DOCUMENTACIÓ DESPLEGAMENT APLICACIONS WEB

Alumnes: Albert Prats, Oriol Valentines i Marc Valentines
Curs: 2n Desenvolupament d'aplicacions Web
Data: Divendres, 1 de maig de 2026

ÍNDEX

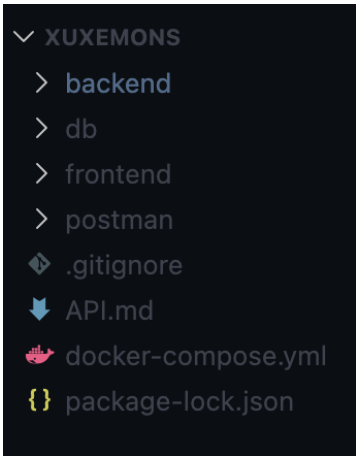
1. CONTENIDORS UTILITZATS 3
2. ARXIU DOCKER COMPOSE 3
2.1 VARIABLES D'ENTORN CONTENIDOR BACKEND 6
2.2 VOLUMS 7
2.3 XARXA 8
2.4 DEPENDÈNCIES 9
2.5 COMPROVACIONS I FUNCIONALITATS EXTRES: 9
2.6 DOCKERFILES 10
2.7 VALIDACIÓ FUNCIONAMENT 12

1. CONTENIDORS UTILITZATS

- Per fer el desplegament d'aquest projecte hem utilitzat tres contenedors diferents, un que emmagatzema la part de frontend (Angular), l'altre emmagatzema el backend de l'aplicació (Laravel) i finalment un contenedor exclusiu que emmagatzema la base de dades (MySQL) on es guarda la informació que gestiona el contenedor del backend.

2. ARXIU DOCKER COMPOSE

- En aquest arxiu configurem els 3 contenidors utilitzats per desplegar la nostra aplicació, també definim la xarxa interna que utilitzaran aquests contenidors per comunicar-se entre si.



Aquesta es la estructura actual del porjecte, els directoris que s'assignen a cada contenidor estàn al mateix nivell que l'arxiu docker compose, per això, per asignar els directoris a "copiar" al docker compose utilitzem **./nom-directori.**

docker-compose.yml

```yaml
name: fullstack-xuxemons
services:
  backend:
    container_name: backend-laravel
    build:
      context: ./backend
      dockerfile: Dockerfile
    ports:
      - "8000:8000"
    environment:
      - DB_HOST=db
      - DB_PORT=3306
      - DB_DATABASE=xuxemons_db
      - DB_USERNAME=xuxemons_admin
      - DB_PASSWORD=1234
    volumes:
      - ./backend:/app
      - /app/vendor
    depends_on:
      db:
        condition: service_healthy
    networks:
      - fullstack-network

  frontend:
    container_name: frontend-angular
    build:
      context: ./frontend
      dockerfile: Dockerfile
    ports:
      - "4200:4200"
    volumes:
      - ./frontend:/app
      - /app/node_modules
    command: npm start -- --host 0.0.0.0 --poll 2000
    depends_on:
      - backend
    networks:
      - fullstack-network
```

```yaml
  db:
    container_name: db-mysql
    build:
      context: ./db
      dockerfile: Dockerfile
    ports:
      - "3306:3306"
    volumes:
      - db_volume:/var/lib/mysql
    networks:
      - fullstack-network
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 5s
      timeout: 5s
      retries: 5
volumes:
  db_volume:

networks:
  fullstack-network:
    driver: bridge
```

name: fullstack-xuxemons - declara el nom del projecte (aquest es veurà al Docker.

Serveis declarats (contenidors que es crearan):
backend: Laravel (backend-laravel)
frontend: Angular (frontend-angular)
db: MySQL (db-sql)

Cada imatge del contenidor es crearà a partir del paràmetre **build**, on es declara que es crearà a partir del **context** (carpeta):

backend → ./backend
frontend → ./frontend
db → ./db

On asginarem amb el paràmetre **dockerfile** l'arxiu **Dockerfile** que es troba dins de cada directori per a que generi la imatge.

Per cada contenidor/servei declarem ports a exposar per poder accedir-hi. En el nostre cas exposem els ports per defecte que utilitzen cada projecte individualment quan es creen, utilitzem el paràmetre **ports** per fer-ho

Ports assignats:
backend (Laravel) → Port 8000:8000
frontend (Angular) → Port 4200:4200
db (MySQL) → Port 3306:3306

2.1 VARIABLES D'ENTORN CONTENIDOR BACKEND

- Aquestes les declarem per a que el contenidor del **backend (Laravel)** pugui accedir a la base de dades que està situada al contidor de **db**, aquestes son les mateixes que trobem al arxiu **.env** del projecte de Laravel.

```
environment:
  - DB_HOST=db
  - DB_PORT=3306
  - DB_DATABASE=xuxemons_db
  - DB_USERNAME=xuxemons_admin
  - DB_PASSWORD=1234
```

DB_HOST: Apunta al contenidor de la base de dades (db), indica on està la base de dades.
DB_PORT: Port on escolta MySQL (per defecte)
DB_DATABASE: Nom de la base de dades a utilitzar
DB_USERNAME: Usuari que s'utilitza per realitzar accions (ha de tenir permisos per actuar sobre la base de dades)
DB_PASSWORD: Contrasenya de l'usuari assignat a la variable **DB_USERNAME**.

2.2 VOLUMS

- Per evitar perdre dades al reiniciar/apagar els contenidors (inclús quan aquests es borren) utilitzem volumns per això. Cada contenidor declarat te un volum creat on s'emmagatzemen diferent contingut, també podem indicar en quina secció volem que s'emmagatzemi, utilitzem el paràmetre **volumes** per fer-ho.

VOLUMS BACKEND:

```
volumes:
  - ./backend:/app
  - /app/vendor
```

- Assignem la carpeta **backend** (on es troba tota l'aplicació de Laravel) a la carpeta /app per a que sigui persistent.
- Volumen anònim per evitar que les dependències de Laravel que tenim en local puguin crear conflictes amb les dependències del contenidor.

VOLUMS FRONTEND:

```
volumes:
  - ./frontend:/app
  - /app/node_modules
```

- Assignem la carpeta **frontend** (on es troba tota l'aplicació d'Angular) a la carpeta /app per a que sigui persistent.
- Volum anònim per evitar que les dependències d'Angular que tenim en local pugin crear conflictes amb les dependències del contenidor.

VOLUM DB:

```
volumes:
  db_volume:
```

- Declarem un volum persistent personalitzat.

```
volumes:
  - db_volume:/var/lib/mysql
```

- Assignem a db_volume que emmagatzemi el contingut del directori **/var/lib/mysql**, aquest emmagatzema tota la informació de la base de dades, ho fem així per a que sigui persistent.

- Utilitzem els directoris on estàn els projectes d'Angular i Laravel com a volums per poder "sincronitzar" directament els canvis que fem amb la carpeta /app del contenidor de Docker.

2.3 XARXA

```
networks:
  fullstack-network:
    driver: bridge
```

- Declarem la xarxa al docker-compose anomenada **fullstack-network**, aquesta permetrà connectar-se entre si als contenidors que tinguin la xarxa, ja que per defecte els contenidors estan aïllats entre si.

2.4 DEPENDÈNCIES

- Per un correcte funcionament del desplegament, hem de crear un “ordre” per a que els contenidors començin a funcionar, per exemple, el contenidor de **backend** depèn d’una base de dades, ja que sinó o no arrancarà, o començarà a donar errors, per assignar dependències dins d’un contenidor/servei, utilitzarem el paràmetre **depends_on**.

```
depends_on:
  db:
    condition: service_healthy
```

Dins del backend, assignem el paràmetre **depends_on** on indiquem que depèn del servei/contenidor **db** i que aquest ha d’estar en una condició saludable (sense errors).

```
depends_on:
  - backend
```

Al contenidor/servei **frontend** assignem que depén del contenidor **backend** per poder iniciar, així que aquest serà l’últim en iniciar-se.

2.5 COMPROVACIONS I FUNCIONALITATS EXTRES:

FRONTEND:

```
  command: npm start -- --host 0.0.0.0 --poll 2000
```

Amb el paràmetre **command** indiquem al contenidor que ha d’executar al arrencar, aquesta commanda ho utilitzem per a que refresqui els canvis cada 2 segons, ja que a un contenidor, els canvis d’Angular no es mostren autimàticament per defecte.

DB:

```
healthcheck:
  test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
  interval: 5s
  timeout: 5s
  retries: 5
```

Per comprovar que la base de dades funciona correctament, fem un ping a la pròpia base de dades per comprovar que hi ha conexió, on es fa 5 vegades, amb un interval de 5 segons, i amb un temps límit de 5 segons per a que la base de dades respongui, una vegada passat el test, es dona el contenidor com a saludable, així el backend pot iniciar.

2.6 DOCKERFILES

DOCKERFILE CONTENIDOR BACKEND:

```
FROM php:8.2-fpm-alpine
RUN docker-php-ext-install pdo pdo_mysql
RUN curl -sS https://getcomposer.org/installer | php -- \ --install-dir=/usr/local/bin --filename=composer
COPY --from=composer:latest /usr/bin/composer /usr/bin/composer
EXPOSE 8000
WORKDIR /app
COPY . .
RUN composer install
CMD ["php", "artisan", "serve", "--host=0.0.0.0", "--port=8000"]
```

- Utilitzem la imatge de PHP 8.2 alpine (lleugera)
- Instal·lació de les extensions que permeten la conexió a la base de dades MySQL.
- Instal·lem la última versió de composer
- Copiem la imatge de composer
- Exposem el port 8000
- Assignem el directori de treball a /app
- Copiem el projecte laravel dins el directori /app
- Executem **composer install** (instal·lem dependències php)
- Executem la comanda per arrancar el servidor de laravel.

DOCKERFILE CONTENIDOR FRONTEND:

```
FROM node:22-alpine
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm install
COPY . .
EXPOSE 4200
CMD ["npm", "start", "--", "--host", "0.0.0.0", "--port", "4200"]
```

- Utilitzem la imatge de node 22 alpine (versió per angular 18 i lleugera)
- Assignem el directori /app com a directori de treball
- Copiem el package.json i package-lock.json (contenen dependencies)
- Executem npm install (instalem dependències)
- Copiem el directori frontend dins de /app
- Exposem port 4200
- Arranquem servidor accessible des de fora del contenidor.

DOCKERFILE CONTENIDOR DB:

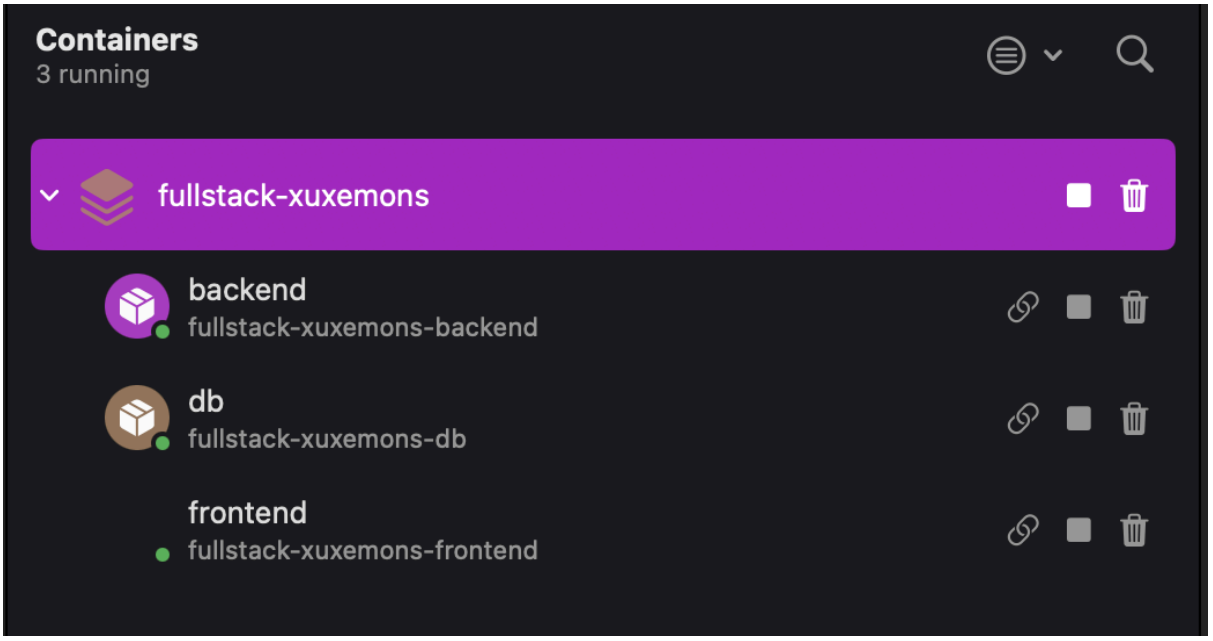
```
FROM mysql:latest
ENV MYSQL_ROOT_PASSWORD=root
ENV MYSQL_DATABASE=xuxemons_db
ENV MYSQL_USER=xuxemons_admin
ENV MYSQL_PASSWORD=1234
```

- Utilitzem la última versió de la imatge de mySQL.
- Assignem les variables de entorn.

2.7 VALIDACIÓ FUNCIONAMENT

EXECUTAR docker compose up –build -d

```
✔ fullstack-xuxemons-db                          Built                                   0.0s
✔ fullstack-xuxemons-backend                     Built                                   0.0s
✔ fullstack-xuxemons-frontend                    Built                                   0.0s
✔ Network fullstack-xuxemons_fullstack-network   Created                                 0.0s
✔ Volume fullstack-xuxemons_db_volume            Created                                 0.0s
✔ Container db-mysql                             Healthy                                12.1s
✔ Container backend-laravel                      Started                                12.1s
✔ Container frontend-angular                     Started                                11.9s
```



EXECUTAR docker ps

```
[Restaurado: 25 abr 2026, 13:22:40]
Last login: Sat Apr 25 13:15:21 on ttys001
→  xuxemons git:(albert) ✗ docker ps
CONTAINER ID   IMAGE                    COMMAND                  CREATED         STATUS                   PORTS                                                   NAMES
897f0911d734   fullstack-xuxemons-frontend   "docker-entrypoint.s…"   3 minutes ago   Up 3 minutes             0.0.0.0:4200->4200/tcp, [::]:4200->4200/tcp             frontend-angular
7e7cd0c4cb24   fullstack-xuxemons-backend    "docker-php-entrypoi…"   3 minutes ago   Up 3 minutes             0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp, 9000/tcp   backend-laravel
b6d9e020b4ab   fullstack-xuxemons-db         "docker-entrypoint.s…"   3 minutes ago   Up 3 minutes (healthy)   0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp, 33060/tcp   db-mysql
→  xuxemons git:(albert) ✗
```

XARXA CREADA

```
xuxemons — albertprba@MacBook-Air-de-Albert — ..oper/xuxe

→  xuxemons git:(albert) ✗ docker network ls
NETWORK ID     NAME                                  DRIVER    SCOPE
0cfac842aaf7   bridge                                bridge    local
6d7654662840   fullstack-xuxemons_fullstack-network  bridge    local
80d44eddd5fd   host                                  host      local
141e28ef230c   none                                  null      local
→  xuxemons git:(albert) ✗
```

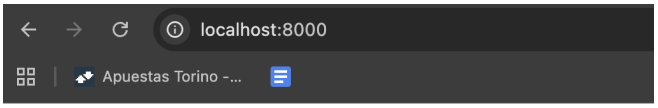
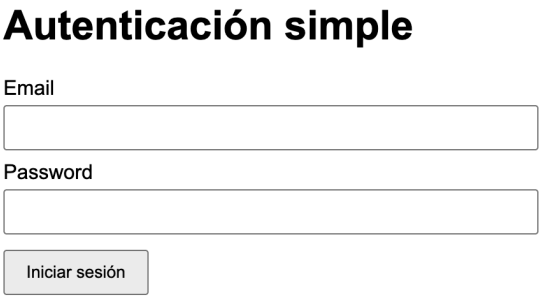
CONTENIDORS CONNECTATS A LA XARXA


```
"Containers": {
    "7e7cd0c4cb24a10b5a22b45d2291d398e2da9eb52f09c447cd9596e4cb782755": {
        "Name": "backend-laravel",
        "EndpointID": "cb4bd00d0eccefac56e9ee12df250dc95dbcaa058a2a4055c4b27ca0208f53b3",
        "MacAddress": "66:9a:70:18:77:d9",
        "IPv4Address": "192.168.97.3/24",
        "IPv6Address": ""
    },
    "897f0911d734bcdb7eb60848c7f022890019fe0069aee25af1b4337b3d992d00": {
        "Name": "frontend-angular",
        "EndpointID": "8e05e1e11d8a7cdd0d2c5779fd9e8dc663b493adb91437af073e13c0301eb7a1",
        "MacAddress": "ae:4f:ef:14:dc:cf",
        "IPv4Address": "192.168.97.4/24",
        "IPv6Address": ""
    },
    "b6d9e020b4abc8c81430455b60b4f4f47b83ed2b328f0564712a7c4421709dca": {
        "Name": "db-mysql",
        "EndpointID": "59a0e591653f1a9f419afebd5f5670fe56be2ffc27e298c69d94555e47abfeea",
        "MacAddress": "92:f0:1d:04:a5:8d",
        "IPv4Address": "192.168.97.2/24",
        "IPv6Address": ""
    }
},
```


PING ENTRE CONTENIDORS (BACKEND A DB I FRONTEND)

```
/app # ping db
PING db (192.168.97.2): 56 data bytes
64 bytes from 192.168.97.2: seq=0 ttl=64 time=0.146 ms
64 bytes from 192.168.97.2: seq=1 ttl=64 time=0.144 ms
64 bytes from 192.168.97.2: seq=2 ttl=64 time=0.144 ms
64 bytes from 192.168.97.2: seq=3 ttl=64 time=0.145 ms
^C
--- db ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 0.144/0.144/0.146 ms
/app #
/app #
/app #
/app #
/app #
/app # ping frontend
PING frontend (192.168.97.4): 56 data bytes
64 bytes from 192.168.97.4: seq=0 ttl=64 time=0.080 ms
64 bytes from 192.168.97.4: seq=1 ttl=64 time=0.188 ms
64 bytes from 192.168.97.4: seq=2 ttl=64 time=0.158 ms
64 bytes from 192.168.97.4: seq=3 ttl=64 time=0.146 ms
^C
--- frontend ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 0.080/0.143/0.188 ms
/app #
```

ACCÉS FORNTEND/BACKEND DESDE NAVEGADOR

FRONTEND:

BACKEND: